



INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

Course: BVWS103-OWASP Top 10 Vulnerabilities And
Exploitation Techniques

Student Name: Abubakari-Sadique Hamidu
Student ID: 2025/FWSD/11392

Instructor: Aminu Iddris (CCNA, CompTIA, CEH, OSCP, CISSP,
CISM)

Title: Lab 1: Advanced SQL Injection Lab Report

Objective

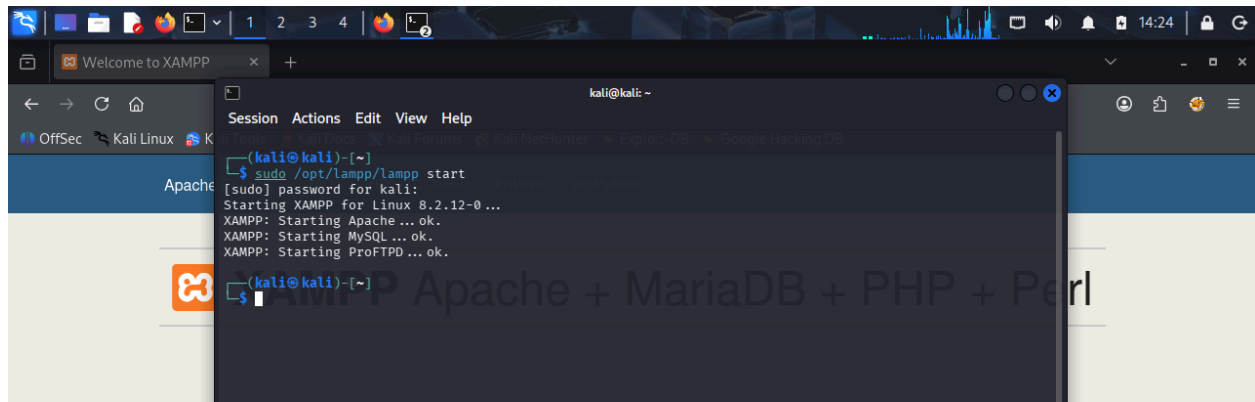
The objective of this lab was to identify and exploit SQL injection vulnerabilities in a deliberately vulnerable web application. The lab focused on error-based SQL injection, time-based blind SQL injection, and second-order SQL injection. The final goal was to apply secure coding practices to mitigate these vulnerabilities.

Lab Environment

- Web Server: XAMPP with Apache, PHP, and MySQL running
- Database: MySQL
- Application Folder: vulnerable_app
- Tools Used: Web browser, phpMyAdmin, code editor, terminal

Step 1: Environment Setup

A local server environment was set up using XAMPP. Apache and MySQL services were started successfully. A database named vulnerable_db was created using phpMyAdmin. The required tables (users and products) were created and populated with sample data.

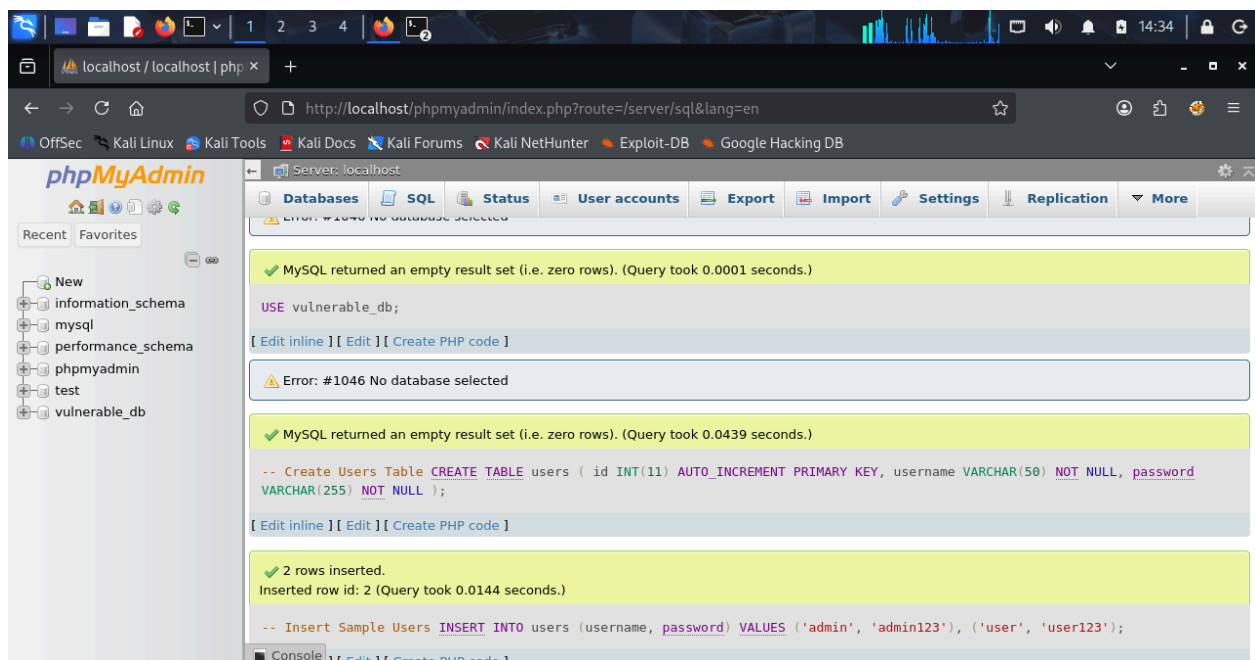


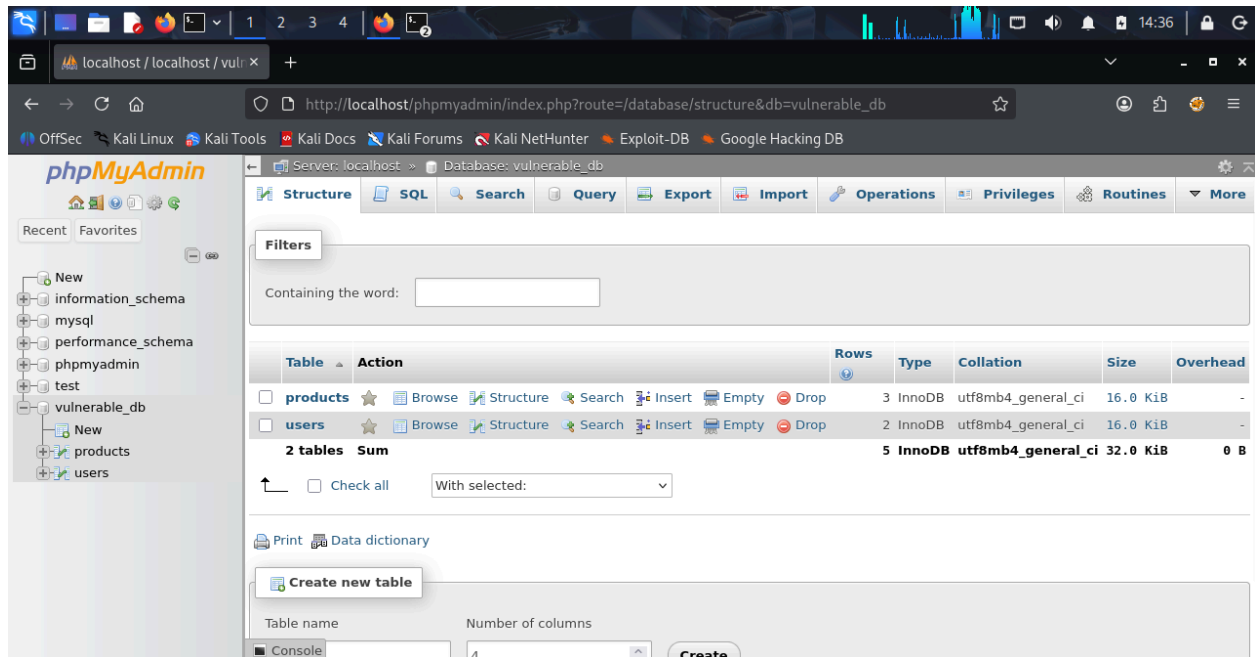
Welcome to XAMPP for Linux 8.2.12

You have successfully installed XAMPP on this system! Now you can start using Apache, MariaDB, PHP and other components. You can find more info in the FAQs section or check the HOW-TO Guides for getting started with PHP applications.

XAMPP is meant only for development purposes. It has certain configuration settings that make it easy to develop locally but that are insecure if you want to have your installation accessible to others.

Start the XAMPP Control Panel to check the server status.



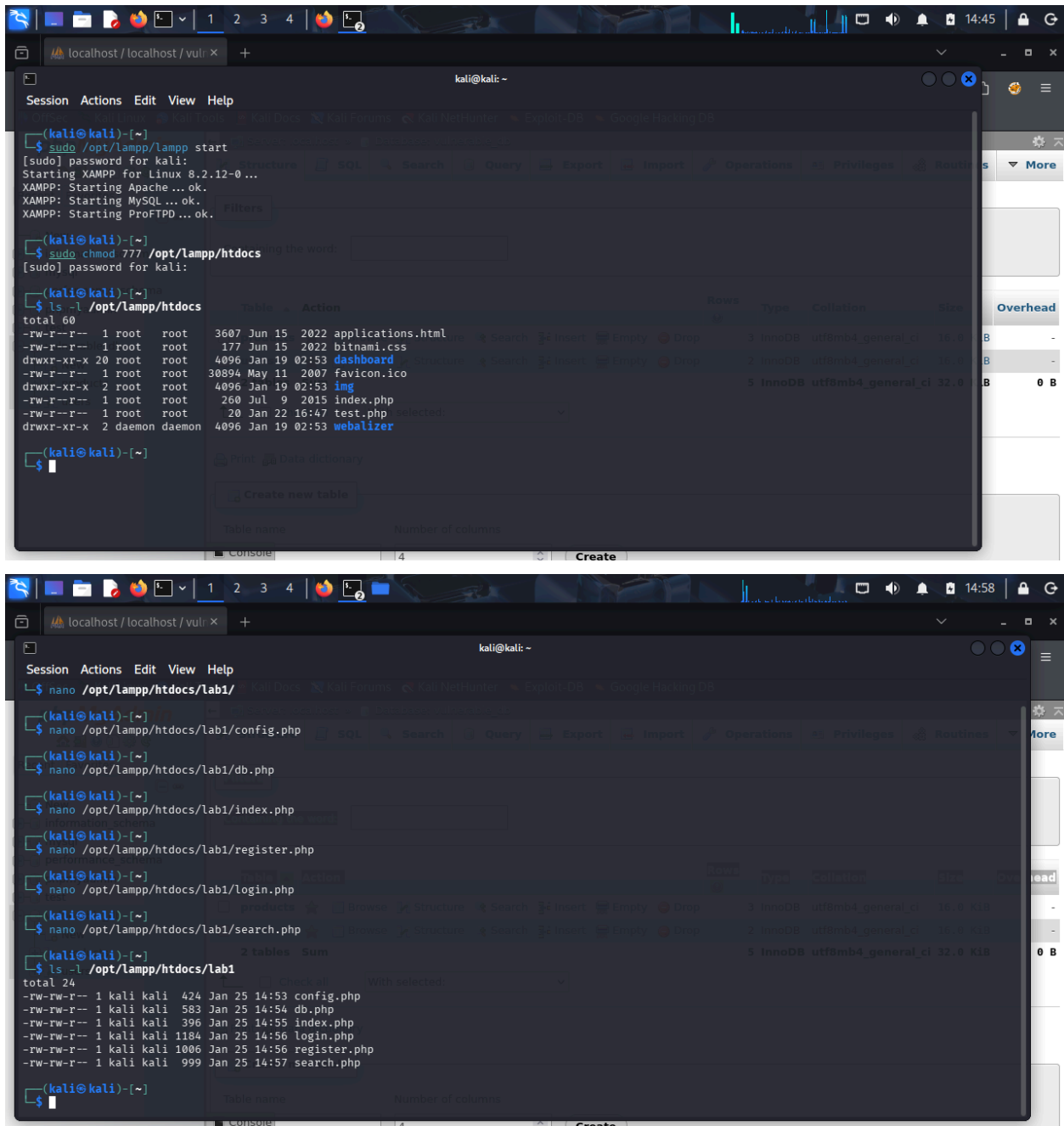


Step 2: Vulnerable Application Setup

Lab1 folder was created inside the htdocs directory. The application consisted of the following files:

- config.php for database connection
- db.php for database queries
- index.php as the home page
- register.php for user registration
- login.php for user authentication
- search.php for product search

The application intentionally used unsanitized user input in SQL queries, making it vulnerable to SQL injection.



Step 3: SQL Injection Exploitation

Error-Based SQL Injection

The search functionality was tested using UNION-based payloads. Error messages and database information were successfully retrieved through the search parameter.

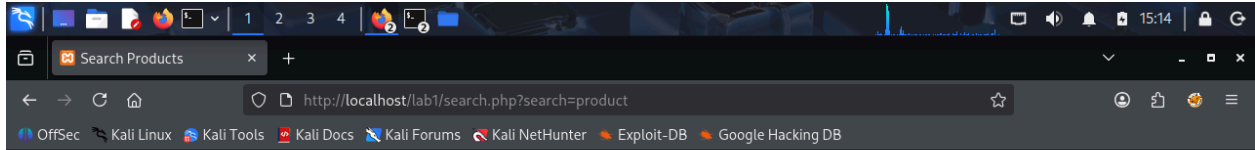
Screenshot 6: Error-based SQL injection payload

Screenshot 7: Database information disclosed via error-based injection

Information Disclosure

The following information was successfully extracted using UNION SELECT payloads:

- Database version
- Current database name
- Current database user
- Table names and column names



Search Results:

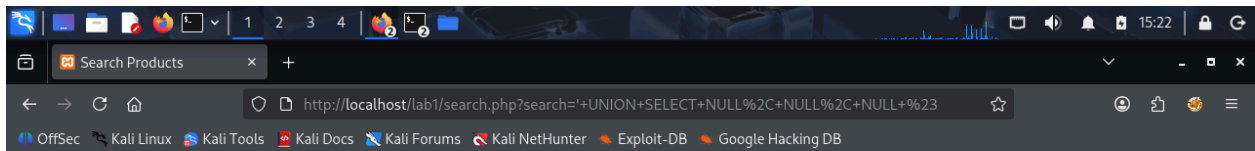
Product: Product 1
Description: Description of Product 1

Product: Product 2
Description: Description of Product 2

Product: Product 3
Description: Description of Product 3

Search for Products

Search:



Search Results:

Product: Product 1
Description: Description of Product 1

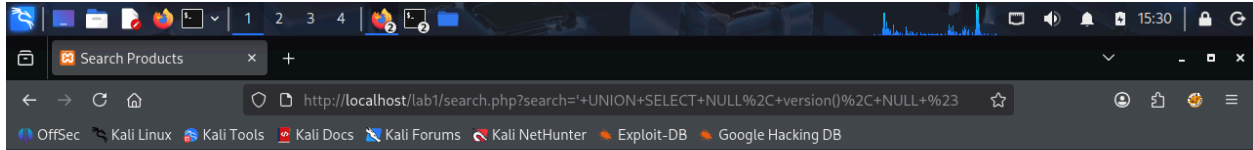
Product: Product 2
Description: Description of Product 2

Product: Product 3
Description: Description of Product 3

Product:
Description:

Search for Products

Search:



Search Results:

Product: Product 1
Description: Description of Product 1

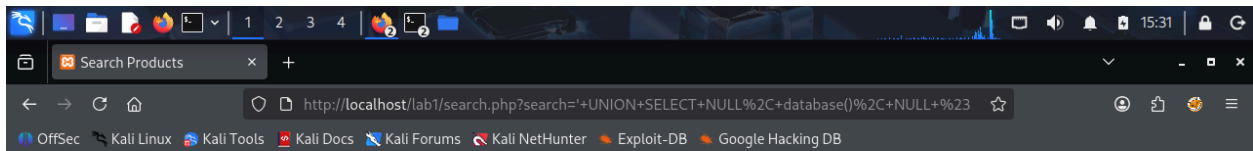
Product: Product 2
Description: Description of Product 2

Product: Product 3
Description: Description of Product 3

Product: 10.4.32-MariaDB
Description:

Search for Products

Search:



Search Results:

Product: Product 1
Description: Description of Product 1

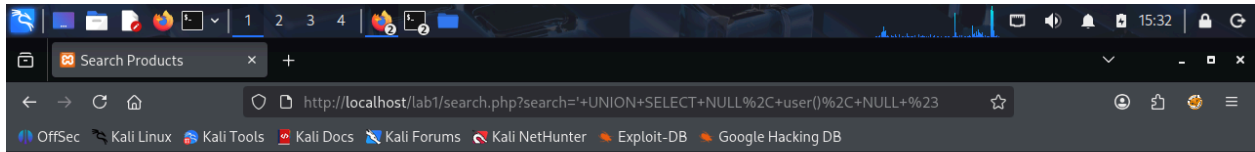
Product: Product 2
Description: Description of Product 2

Product: Product 3
Description: Description of Product 3

Product: vulnerable_db
Description:

Search for Products

Search:



Search Results:

Product: Product 1
Description: Description of Product 1

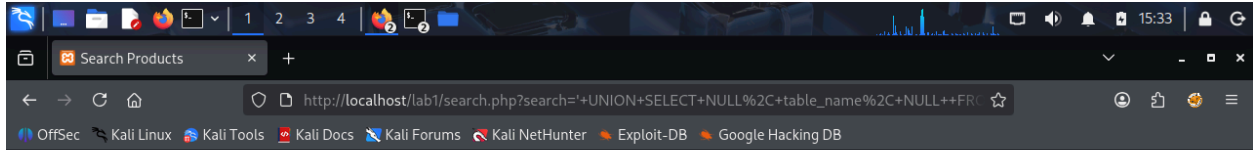
Product: Product 2
Description: Description of Product 2

Product: Product 3
Description: Description of Product 3

Product: root@localhost
Description:

Search for Products

Search:



Search Results:

Product: Product 1
Description: Description of Product 1

Product: Product 2
Description: Description of Product 2

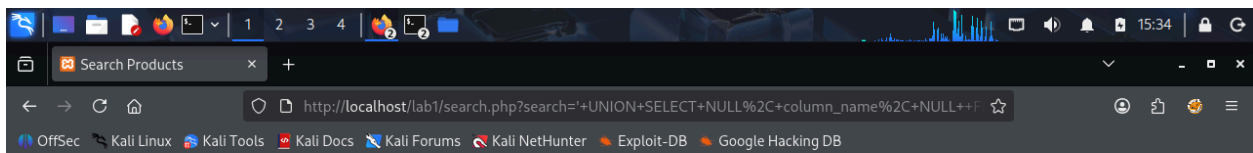
Product: Product 3
Description: Description of Product 3

Product: products
Description:

Product: users
Description:

Search for Products

Search:



Product: Product 1
Description: Description of Product 1

Product: Product 2
Description: Description of Product 2

Product: Product 3
Description: Description of Product 3

Product: id
Description:

Product: username
Description:

Product: password
Description:

Product: USER
Description:

Product: CURRENT_CONNECTIONS
Description:

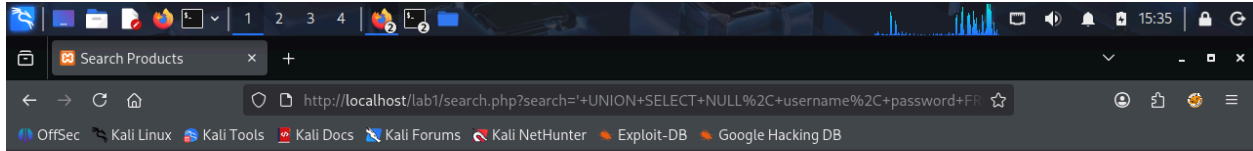
Product: TOTAL_CONNECTIONS
Description:

Time-Based Blind SQL Injection

The login functionality was tested using a time-based payload with the SLEEP(5) function. A noticeable delay confirmed the vulnerability.

Second-Order SQL Injection

Malicious input was stored during user registration and later executed when retrieved from the database, demonstrating second-order SQL injection.



Search Results:

Product: Product 1
Description: Description of Product 1

Product: Product 2
Description: Description of Product 2

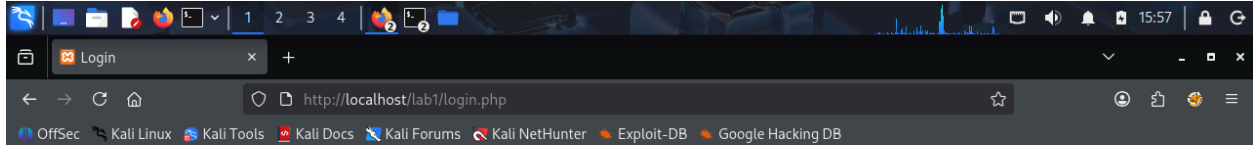
Product: Product 3
Description: Description of Product 3

Product: admin
Description: admin123

Product: user
Description: user123

Search for Products

Search:



Login successful!

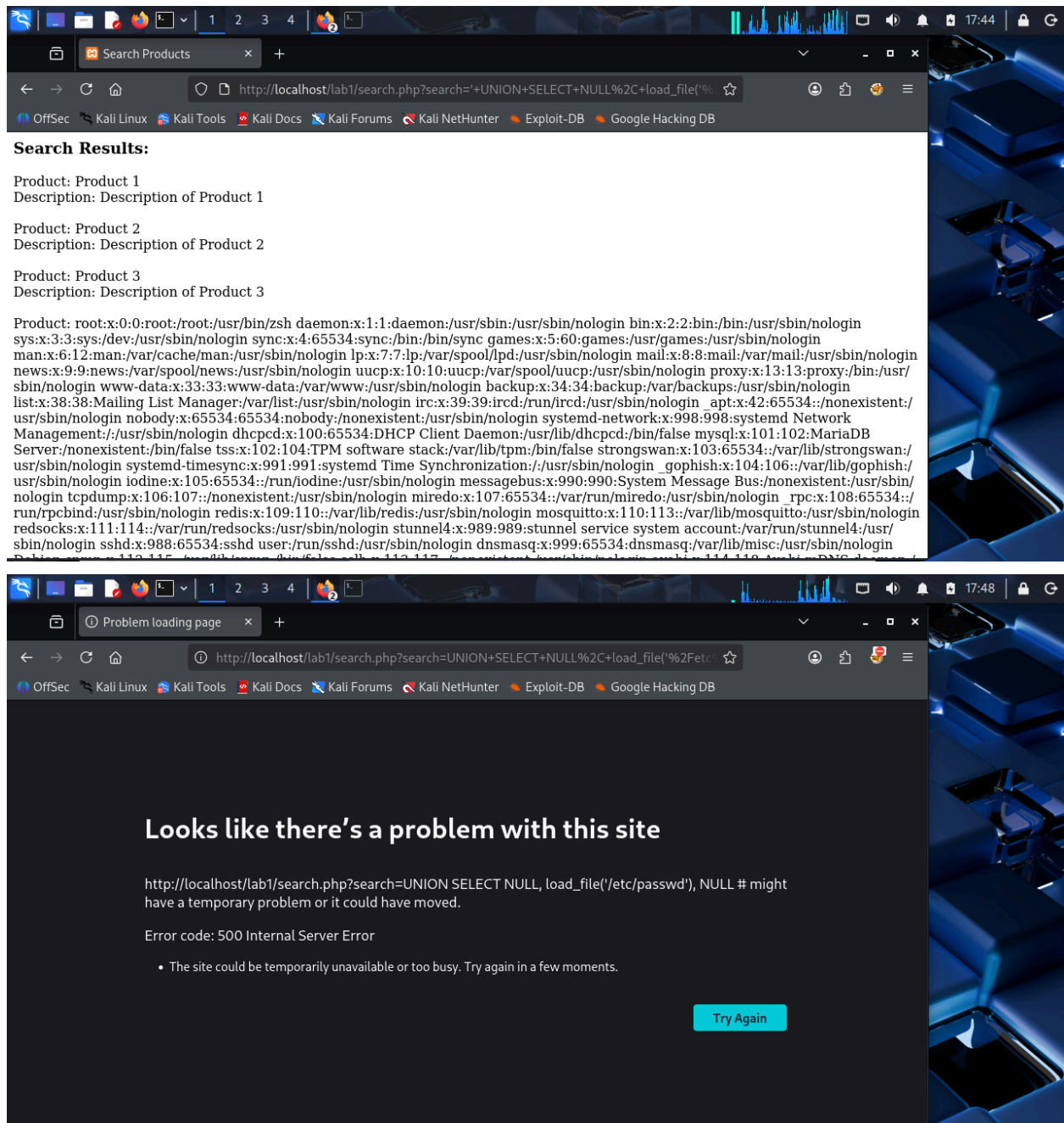
Login

Username:

Password:

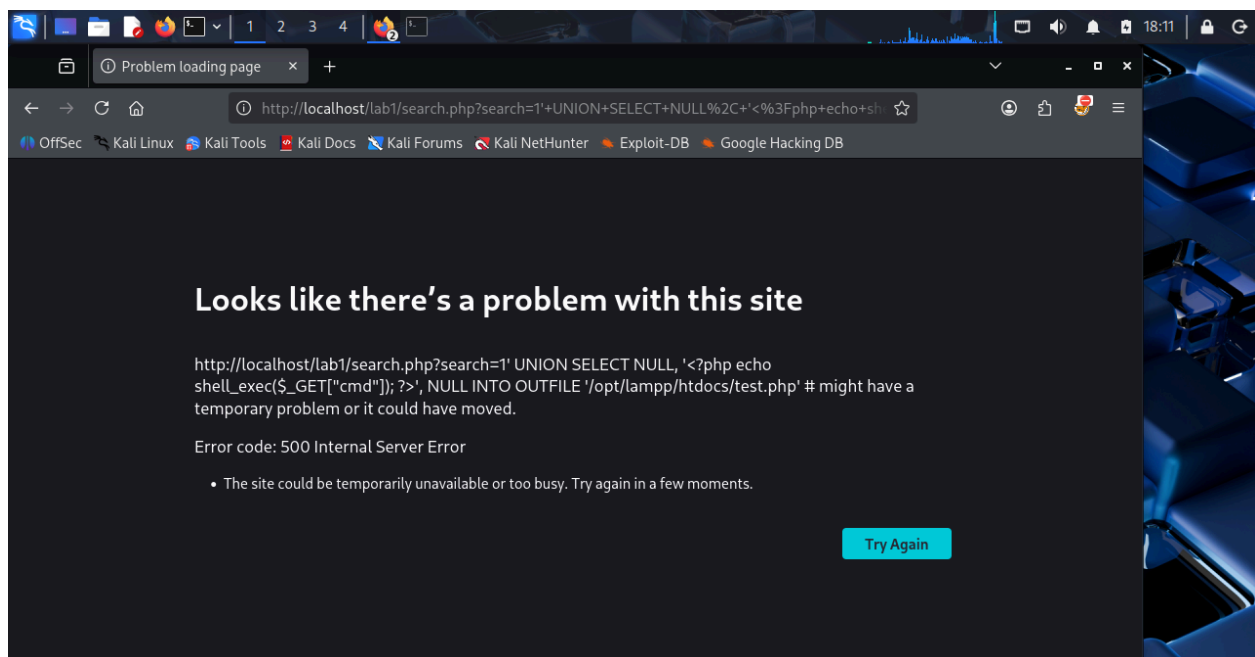
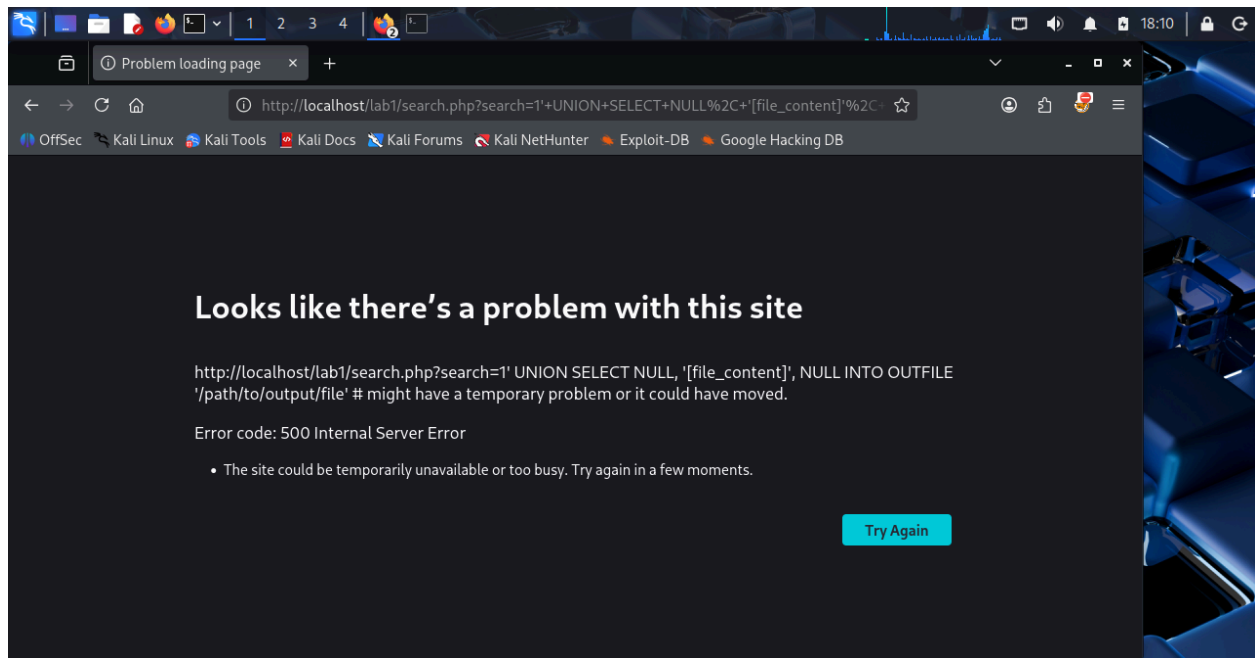
Scenario 7: File Read Using LOAD_FILE

Attempts were made to read system files using the LOAD_FILE function. The server returned errors due to restricted FILE privileges. This behavior confirmed that file read attacks were mitigated at the database configuration level.



Scenario 8: File Write Using OUTFILE

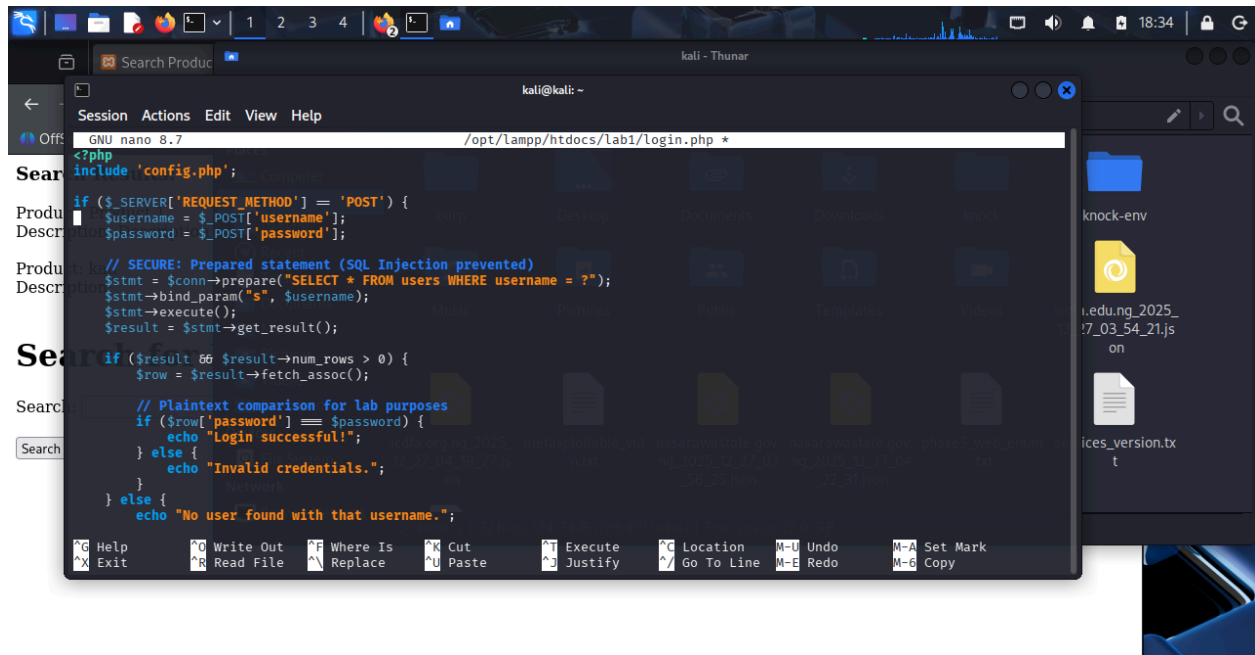
Attempts to write files to the server using INTO OUTFILE were unsuccessful. The server returned HTTP 500 errors, indicating that MySQL FILE privileges and directory permissions prevented file write operations.



Step 4: Mitigation Implementation

Prepared Statements

Prepared statements were implemented in login.php to replace vulnerable SQL queries. User input was parameterized, preventing SQL injection.



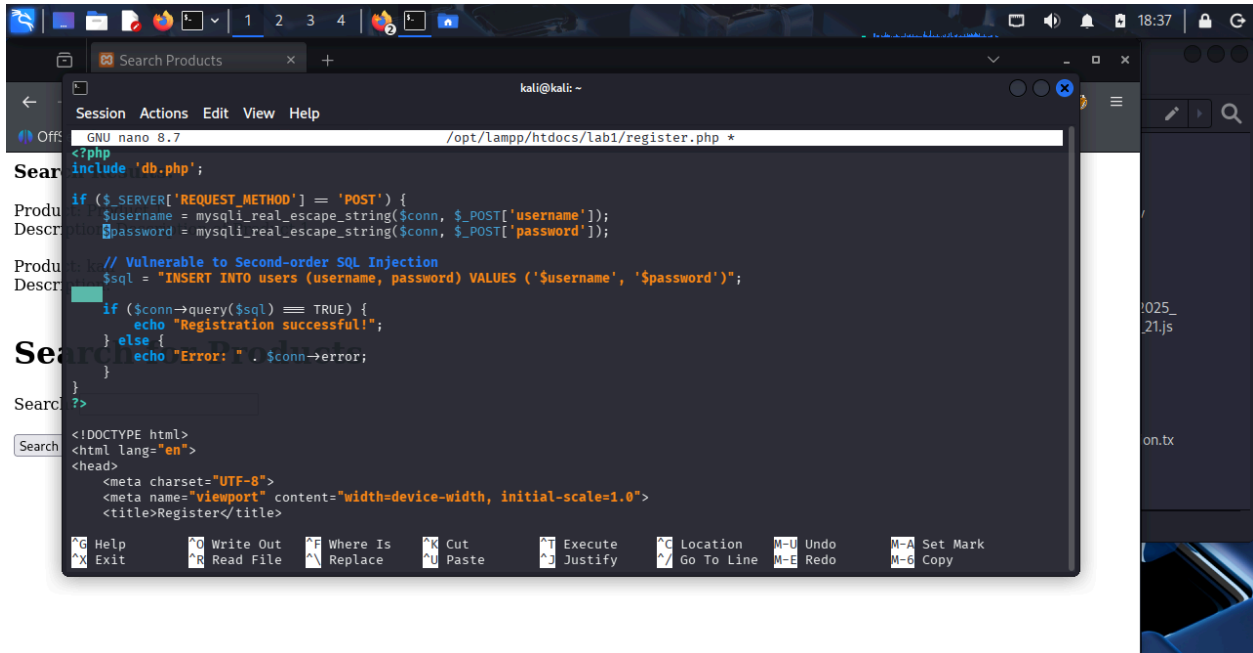
```
GNU nano 8.7 /opt/lampp/htdocs/lab1/login.php *
<?php
include 'config.php';

if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $username = $_POST['username'];
    $password = $_POST['password'];

    // SECURE: Prepared statement (SQL Injection prevented)
    $stmt = $conn->prepare("SELECT * FROM users WHERE username = ?");
    $stmt->bind_param("s", $username);
    $stmt->execute();
    $result = $stmt->get_result();

    if ($result->num_rows > 0) {
        $row = $result->fetch_assoc();

        // Plaintext comparison for lab purposes
        if ($row['password'] == $password) {
            echo "Login successful!";
        } else {
            echo "Invalid credentials.";
        }
    } else {
        echo "No user found with that username.";
    }
}
```



```
GNU nano 8.7 /opt/lampp/htdocs/lab1/register.php *
<?php
include 'db.php';

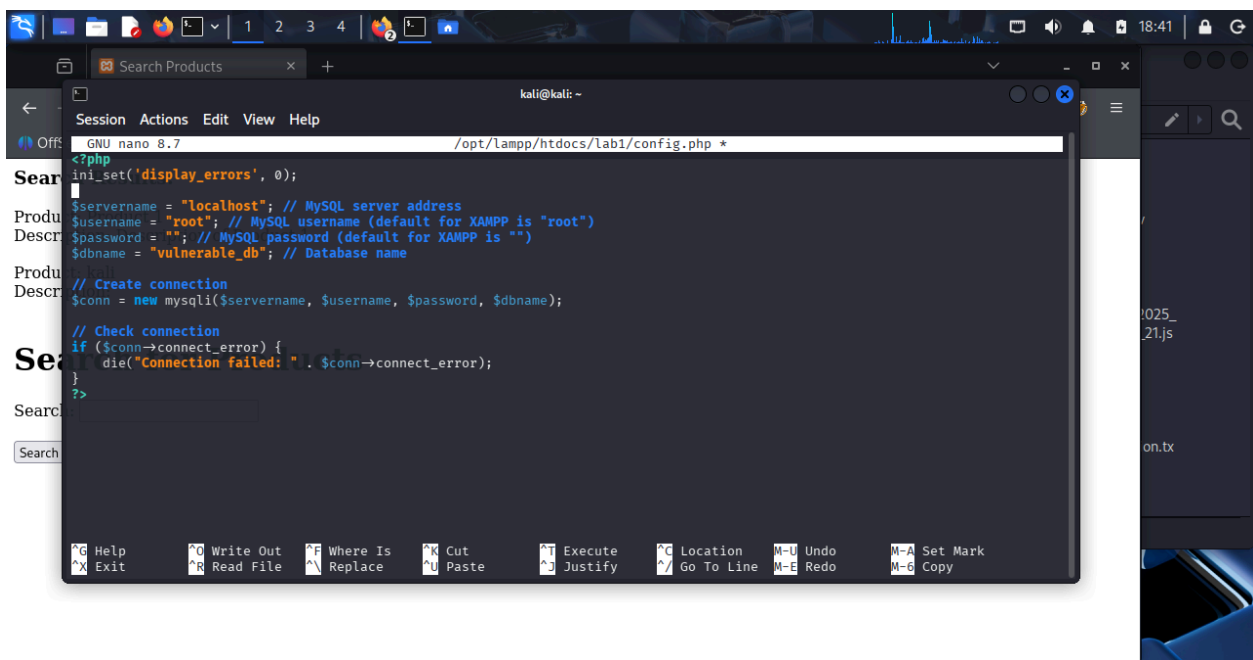
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $username = mysqli_real_escape_string($conn, $_POST['username']);
    $password = mysqli_real_escape_string($conn, $_POST['password']);

    // Vulnerable to Second-order SQL Injection
    $sql = "INSERT INTO users (username, password) VALUES ('$username', '$password')";

    if ($conn->query($sql) == TRUE) {
        echo "Registration successful!";
    } else {
        echo "Error: " . $conn->error;
    }
}

?>

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Register</title>
```



```
GNU nano 8.7 /opt/lampp/htdocs/lab1/config.php *
<?php
ini_set('display_errors', 0);

$servername = "localhost"; // MySQL server address
$username = "root"; // MySQL username (default for XAMPP is "root")
$password = ""; // MySQL password (default for XAMPP is "")
$dbname = "vulnerable_db"; // Database name

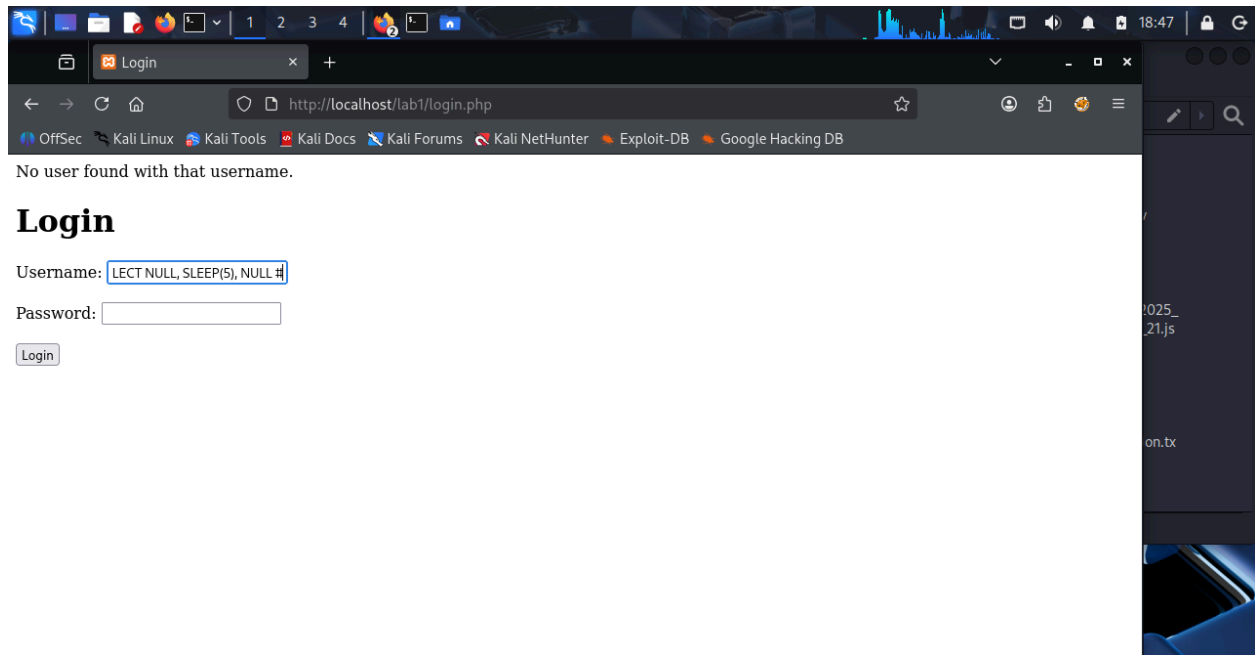
// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);

// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

?>
```

Retesting Injection

The same time-based SQL injection payload using SLEEP(5) was tested after mitigation. No delay was observed, confirming that the vulnerability was successfully fixed.



Additional Mitigations

- Input sanitization was applied to registration inputs
- Error reporting was disabled to prevent information leakage
- Database permissions were kept restrictive

Conclusion

This lab demonstrated how SQL injection vulnerabilities can be exploited to extract sensitive information and potentially compromise a system. It also showed how proper mitigation techniques such as prepared statements, input validation, and secure configuration can effectively prevent SQL injection attacks. The lab reinforced the importance of secure coding practices in real-world applications.